

RetinaScan: Segmentación de Vasos Sanguíneos en Retina con Arquitectura U-Net

Marcos Ángel Ayala Blanco

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
marayabla@alum.us.es

Lorenzo Valderrama Román

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
lorvalrom@alum.us.es

Resumen—Este trabajo presenta el diseño, entrenamiento y evaluación de una red neuronal con arquitectura U-Net para la segmentación automática de vasos sanguíneos en imágenes de fondo de ojo. El objetivo es construir un sistema capaz de marcar, píxel a píxel, dónde se encuentran los vasos sanguíneos en fotografías médicas reales, una tarea que habitualmente realiza un médico especialista de forma manual. Para ello, se ha trabajado con la base de datos pública DRIVE 2004 [7], aplicando técnicas de preprocesado con CLAHE, aumento de datos dinámico y validación cruzada con 5 pliegues.

Los resultados obtenidos superan el umbral mínimo exigido, con una puntuación media DICE de 0,7746 sobre el conjunto de test completo, evaluada frente a las anotaciones de dos expertos. El trabajo demuestra que es posible entrenar un modelo de este tipo de forma eficiente, gracias a un diseño cuidadoso del flujo de datos, un correcto preprocesado de las imágenes y un postprocesado basado en umbralización adaptativa y operaciones morfológicas.

Palabras clave—Redes neuronales convolucionales (CNN), U-Net, segmentación semántica binaria, retina, DRIVE 2004, deep learning, Keras, coeficiente DICE, BCE-DICE Loss, visión por computador, aumento de datos, validación cruzada, desbalanceo de clases, CLAHE, Test-Time Augmentation (TTA), algoritmo de Otsu.

I. INTRODUCCIÓN

En medicina, el análisis de las imágenes del fondo de ojo es una herramienta muy valiosa para detectar enfermedades como la diabetes, la hipertensión o problemas cardiovasculares [7]. Para ello, un médico especialista tiene que observar la fotografía de la retina e identificar manualmente dónde están los vasos sanguíneos y si presentan alguna anomalía. Este proceso es lento, costoso y requiere mucha experiencia clínica. Aquí es donde la inteligencia artificial puede marcar una diferencia real.

En los últimos años, las redes neuronales convolucionales (CNN) [1], [2] han demostrado ser muy eficaces analizando imágenes. Dentro de este campo, la arquitectura U-Net [3] destaca porque no solo detecta qué hay en una imagen, sino que además es capaz de señalar exactamente *dónde* está cada estructura, trabajando a nivel de píxel individual. Esto la convierte en la herramienta idónea para tareas médicas donde la precisión espacial es fundamental.

El objetivo de este trabajo es entrenar una U-Net para que aprenda a marcar automáticamente los vasos sanguíneos en imágenes de retina, a partir de ejemplos previamente anotados

por médicos expertos (a los que nos referiremos a lo largo del proyecto como "ground truth"). Una vez entrenado, el modelo puede analizar una imagen nueva en milisegundos y generar una máscara binaria indicando con exactitud qué píxeles corresponden a vasos sanguíneos. Esto supone un paso hacia la automatización del diagnóstico médico, con el potencial de asistir a los profesionales sanitarios en su trabajo diario.

El resto del documento se organiza de la siguiente manera: la Sección II describe las técnicas empleadas y el trabajo relacionado; la Sección III detalla el conjunto de datos y el problema; la Sección IV explica la métrica de evaluación; la Sección V describe las decisiones de diseño e implementación; la Sección VI presenta los resultados; y la Sección VII recoge las conclusiones.

II. PRELIMINARES

A. Técnicas empleadas

El núcleo del proyecto es una **Red Neuronal Convolucional (CNN)** [1], un tipo de arquitectura de aprendizaje profundo pensada para procesar imágenes. A diferencia de una red neuronal tradicional, las CNN utilizan filtros que se desplazan sobre la imagen detectando patrones visuales (bordes, texturas, formas) de forma automática. Esto las hace especialmente eficaces para tareas de visión por computador.

Dentro de las CNN, la **arquitectura U-Net** [3] fue diseñada específicamente para segmentación de imágenes biomédicas. Su nombre viene de su forma en "U": primero comprime la imagen extrayendo información (encoder) y luego la expande de vuelta a su tamaño original para localizar cada estructura con precisión (decoder). El elemento clave que la diferencia de otras arquitecturas son las **conexiones de salto** (*skip connections*), que transfieren información de alta resolución del encoder al decoder, permitiendo generar máscaras con bordes muy precisos incluso en estructuras finas como los vasos capilares.

B. Validación Cruzada con K-Fold

La **validación cruzada con K pliegues** (*K-Fold Cross-Validation*) [14] es una técnica estadística que permite evaluar y entrenar modelos de aprendizaje automático de forma robusta cuando el conjunto de datos disponible es reducido. En

lugar de hacer una única división fija entre datos de entrenamiento y validación, el dataset se divide en K subconjuntos o *pliegues* de igual tamaño. El proceso se repite K veces: en cada iteración, uno de los pliegues actúa como conjunto de validación mientras los $K - 1$ restantes se usan para entrenar. Al finalizar las K iteraciones se obtienen K modelos independientes, y la métrica final se calcula como la media de los K resultados.

Esta técnica tiene dos ventajas fundamentales frente a una partición única. En primer lugar, **todos los ejemplos participan tanto en entrenamiento como en validación**, lo que aprovecha al máximo un dataset pequeño. En segundo lugar, la media de los K resultados es una estimación mucho más estable del rendimiento real del modelo que una sola evaluación, ya que reduce el impacto de haber elegido una partición concreta por azar. En este trabajo se ha utilizado **5-Fold** ($K = 5$), dividiendo las 20 imágenes de entrenamiento de DRIVE en 5 grupos de 4.

C. Estado del Arte y Trabajo Relacionado

La segmentación de vasos sanguíneos en la retina es un problema clínico que ha sido ampliamente estudiado. Al revisar el **estado del arte** previo a la revolución computacional, observamos que los métodos clásicos se basaban en filtros de Frangi (detectores de estructuras tubulares), umbralizaciones locales y operaciones morfológicas. Aunque funcionales, estos métodos eran excesivamente sensibles a las variaciones de iluminación y requerían un ajuste manual de los parámetros para cada fotografía [7].

Actualmente, el estado del arte ha dado un salto cualitativo indiscutible con la consolidación del aprendizaje profundo (*deep learning*). La arquitectura U-Net [3] se convirtió rápidamente en el estándar de oro para esta tarea. Trabajos como el de Abdusamatov [4] y otras implementaciones validadas por la comunidad de ciencia de datos [17], [18] demostraron que una U-Net entrenada con la base de datos DRIVE supera de forma drástica y consistente a los métodos clásicos de visión por computador.

Si bien es cierto que el estado del arte más vanguardista de hoy en día explora variantes más complejas (como la *Attention U-Net* o los modelos basados en *Transformers*) para rascar unas milésimas de precisión en casos difíciles, lo hacen a costa de un coste computacional gigantesco. Por ello, en este proyecto se ha optado por implementar la arquitectura U-Net original, logrando un equilibrio óptimo y realista entre el alto rendimiento clínico y la viabilidad de entrenamiento en hardware convencional.

III. DATASET Y OBJETIVO

A. La base de datos DRIVE 2004

Para entrenar y evaluar el modelo se ha utilizado la base de datos **DRIVE 2004** (*Digital Retinal Images for Vessel Extraction*) [7]. Este es un conjunto de datos público y uno de los estándares más utilizados en investigación médica para este tipo de tareas. En total, la base de datos parte de **40 fotografías originales** del fondo de ojo con una resolución de

565×584 píxeles, tomadas mediante un retinógrafo a pacientes con retinopatía diabética en los Países Bajos.

A nivel de organización de archivos, el conjunto de datos se divide de forma estricta en dos grandes bloques (*training* y *test*), sumando un total exacto de **140 archivos** si desglosamos las fotografías en color, las anotaciones de los expertos y las máscaras estructurales:

- **Conjunto de Entrenamiento (60 archivos):** Alojado en la carpeta `data/training`. Se utilizan exclusivamente para enseñar a la red neuronal. Dentro de este bloque encontramos la siguiente estructura:
 - `images/`: Contiene las **20 fotografías originales** de los pacientes en formato `.tif`.
 - `1st_manual/`: Contiene **20 segmentaciones manuales** en formato `.gif` (una por imagen), dibujadas por un experto médico. Esta carpeta actúa como la verdad absoluta (*Ground Truth*) durante el aprendizaje de la red.
 - `mask/`: Contiene **20 máscaras** del campo de visión (FOV) en formato `.gif`.
- **Conjunto de Test (80 archivos):** Alojado en la carpeta `data/test`. Se utilizan para examinar a la red una vez entrenada. Su estructura aporta más información para evaluar al modelo:
 - `images/`: Contiene las **20 fotografías originales** de prueba en formato `.tif`.
 - `mask/`: Contiene las **20 máscaras** del campo de visión (FOV) correspondientes.
 - `1st_manual/` y `2nd_manual/`: A diferencia del entrenamiento, aquí encontramos **40 segmentaciones manuales en total** (20 en la primera carpeta y 20 en la segunda). Estas han sido trazadas de forma independiente por dos oftalmólogos distintos, lo que nos permite medir la variabilidad de opiniones entre médicos.

Las máscaras del **Campo de Visión (FOV)** (alojadas en las carpetas `mask/`) son imágenes en blanco y negro que indican qué zona corresponde al interior del ojo. Como las fotografías médicas son rectangulares pero el ojo es esférico, las esquinas de la imagen son completamente negras y no contienen información. Aplicar esta máscara nos permite decirle al código que ignore esas esquinas al calcular las métricas, evitando que el modelo obtenga una puntuación artificialmente alta por acertar píxeles que obviamente son fondo.

B. Definición del problema

El problema que aborda este proyecto es la **segmentación semántica binaria**. A diferencia de la clasificación de imágenes, que asigna una única etiqueta global a toda la foto, aquí la red analiza cada píxel y decide si pertenece a un vaso sanguíneo (valor 1) o no (valor 0). La Tabla I resume las diferencias clave entre ambos enfoques.

Para la realización de este proyecto, fue fundamental entender la diferencia entre **Clasificación** y **Segmentación**. Mientras que la clasificación de imágenes es el "qué", trata

a la imagen como un todo único, la red neuronal mira la foto completa y le asigna una sola etiqueta global, consiguiendo el objetivo de responder a preguntas como ¿Qué hay en esta imagen? o ¿A qué categoría pertenece?; la segmentación de imágenes es el “dónde” y la “forma”, va a un nivel de detalle microscópico haciendo una clasificación pixel por pixel, alcanzando el objetivo de responder a la pregunta ¿Exactamente qué píxeles pertenecen al objeto que estoy buscando?

En nuestro caso (segmentación), cuando le asignamos a la IA la imagen del ojo, esta examinará los parches de 128x128 píxeles, decidiendo para cada uno de ellos si es “Vaso Sanguíneo”(1) o “Fondo oscuro”(0).

TABLA I
COMPARATIVA ENTRE CLASIFICACIÓN Y SEGMENTACIÓN DE IMÁGENES

Aspecto	Clasificación	Segmentación
Enfoque	Toda la imagen	Píxel a píxel
Salida	Una etiqueta	Una máscara binaria
Complejidad	Baja / Media	Alta
Arquitectura	CNN simple	U-Net
Métrica	Accuracy	DICE score

Este problema predictivo se enfrenta a **dos grandes dificultades clínicas**:

- 1) **Desbalanceo extremo de clases:** Los vasos sanguíneos son muy finos y apenas ocupan entre el 10% y el 15% del área total de la retina. Si nuestro modelo hiciera trampa y predijera que absolutamente toda la imagen es fondo negro, acertaría el 85% de los píxeles. Esto invalida el uso de la métrica clásica de *Accuracy* y nos obliga a usar métricas de solapamiento espacial.
- 2) **Bajo contraste anatómico:** Los capilares de la periferia del ojo miden apenas 1 o 2 píxeles de grosor y se difuminan con el fondo debido a la mala iluminación de las fotografías médicas. Esta falta de contraste genera tal nivel de ambigüedad que ni siquiera los propios oftalmólogos se ponen de acuerdo al trazar la máscara manual, lo que añade un extra de dificultad para que la inteligencia artificial sepa dónde está el límite exacto de la vena.

1) *Objetivos del Proyecto:* El objetivo final del proyecto es diseñar, entrenar y post-procesar un modelo U-Net capaz de emitir máscaras binarias que sean útiles desde un punto de vista clínico, superando un rendimiento de corte establecido por la asignatura.

En concreto, se espera que el modelo entrenado supere como mínimo un **DICE score de 0,75 de media** sobre el conjunto de test completo. Como el conjunto de test incluye las segmentaciones manuales de dos profesionales distintos, esta media se calcula teniendo en cuenta ambas anotaciones. Esto significa que no hay una nota mínima obligatoria individual para cada experto, sino que se evalúa el promedio. Por ejemplo, si nuestro modelo logra una puntuación de 0,78 respecto a la segmentación del primer experto, y un 0,74 respecto a la del segundo experto, tendría una media total de 0,76 y el objetivo sería dado por bueno.

IV. MÉTRICAS

A. Por qué no usamos Accuracy

La medida más sencilla es el *accuracy*: el porcentaje de píxeles clasificados correctamente. Sin embargo, resulta engañosa en este problema. Los vasos sanguíneos solo ocupan alrededor del 10–15% de los píxeles; el resto es fondo. Un modelo que dijera siempre “esto es fondo” obtendría un *accuracy* superior al 85% sin detectar ni un solo vaso, lo que sería completamente inútil para el diagnóstico médico.

B. El coeficiente DICE

Para evitar este problema se utiliza el **coeficiente DICE** [4], [10], que mide directamente el **solapamiento** entre la máscara predicha y la anotación real, centrándose solo en los píxeles donde aparecen los vasos. Su fórmula es:

$$\text{DICE}(P, G) = \frac{2 \cdot |P \cap G|}{|P| + |G|} \quad (1)$$

donde P es el conjunto de píxeles predichos como vaso y G es el *Ground Truth* del médico. El valor oscila entre 0 (ningún parecido) y 1 (coincidencia perfecta). La métrica penaliza tanto los falsos positivos (marca vaso donde no lo hay) como los falsos negativos (no detecta un vaso real).

C. La función de pérdida BCE-DICE

Durante el entrenamiento, la red se guía por una función de pérdida combinada llamada **BCE-DICE Loss**, que une la *Binary Cross-Entropy* (penaliza errores píxel a píxel) con el DICE Loss (penaliza el solapamiento global). Esta combinación hace el entrenamiento más estable y ayuda al modelo a aprender los detalles de los vasos más finos, que de otro modo podrían ignorarse por su escaso peso numérico en el total de píxeles.

V. METODOLOGÍA

A. Preparación de los datos: la clase DataGenerator (Pre-procesado)

Entrenar un modelo de aprendizaje profundo con imágenes de tan alta resolución exige una enorme capacidad de memoria RAM, lo que dificulta el uso de hardware convencional. Para superar este obstáculo, se implementó una clase *DataGenerator* que hereda directamente de `keras.utils.PyDataset`.

El uso de esta superclase (el estándar moderno en Keras 3 [6], [8]) garantiza que la carga de datos sea segura en entornos de multiprocesamiento (*thread-safety*). Nuestra clase actúa como el motor de alimentación del proyecto mediante la técnica de **Carga Diferida** (*Lazy Loading*): en lugar de procesar y volcar todas las retinografías en memoria al iniciar el programa, el generador solo extrae, recorta y procesa un pequeño lote de imágenes (en nuestro caso, `batch_size = 4`) en el instante exacto en el que la red neuronal lo solicita, liberando la memoria inmediatamente después.

A nivel arquitectónico, la clase se compone de tres bloques fundamentales:

- `__init__` (Constructor): Inicializa el estado, recibiendo las rutas de las imágenes y definiendo los tamaños de lote y de recorte.
- `__len__`: Calcula dinámicamente cuántos pasos exactos debe dar la red neuronal para completar una época de entrenamiento (dividiendo el total de imágenes entre el tamaño del lote).
- `__getitem__`: Es el corazón del motor de datos. Se ejecuta cada vez que la U-Net pide un nuevo lote. Recibe un índice y devuelve las matrices X (imágenes) e Y (máscaras) exactas para ese paso.

Dentro del método `__getitem__`, se desarrolla una ingeniería de datos en 6 fases secuenciales, cuyo pseudocódigo se resume en el Alg. 1:

- 1) **Lectura Diferenciada:** Las fotografías del ojo original se encuentran en formato `.tif`, mientras que las máscaras del médico se alojan en `.gif`. Dado que OpenCV soporta la lectura de ambos, se utiliza `cv2.imread`. OpenCV carga los canales de color en orden BGR por motivos de retrocompatibilidad, por lo que se aplica una conversión inmediata a RGB.
- 2) **Filtro CLAHE:** Antes de normalizar, se extraen las venas ocultas en la oscuridad de la retina pasando la imagen al espacio de color LAB. El algoritmo CLAHE [11], [19] se aplica **exclusivamente sobre el canal de luminosidad (L)**, una técnica que ha demostrado gran eficacia en estudios y foros paralelos de segmentación [16]. Este detalle es vital para no alterar ni distorsionar los tonos rojizos originales de los canales cromáticos A y B. Este filtro fue un descubrimiento decisivo que mejoró considerablemente los resultados de nuestros modelos.
- 3) **Normalización Matemática:** Las imágenes digitales poseen valores de píxel entre 0 y 255. Introducir magnitudes tan dispares desestabilizaría el gradiente descendente de la red. Por ello, ambas matrices completas se dividen entre 255,0, comprimiendo los valores a una escala decimal continua entre $[0,0, 1,0]$, lo que permite que la U-Net converja mucho más rápido.
- 4) **Parcheado Espacial (Slicing):** La resolución original (584×565) es incompatible con nuestra arquitectura. Sin embargo, redimensionar la imagen deformaría y aplastaría los diminutos vasos capilares de la retina. La solución fue el parcheado: se calcula un límite de seguridad dinámico restando 128 píxeles (el tamaño del parche) al ancho y alto original. Se generan coordenadas aleatorias (x, y) y se extrae una submatriz de 128×128 exactamente en las mismas coordenadas para la imagen y la máscara, garantizando la alineación espacial.
- 5) **Aumento de Datos al vuelo (Data Augmentation):** Con solo 20 imágenes de entrenamiento, el riesgo de *Overfitting* (que la red memorice las fotos en lugar de aprender) es crítico. Para multiplicar virtualmente nuestro conjunto de datos, aplicamos transformaciones aleatorias en tiempo de ejecución. Por cada parche que se extrae, se decide de forma aleatoria (con un 50%

de probabilidad) si se le aplican volteos (horizontales y verticales) y rotaciones. Con esa misma probabilidad del 50%, se decide también si introducir ruido gaussiano y variaciones de brillo.

- **Protección Matemática (`np.clip`):** Al aplicar brillo a un píxel normalizado (ej. $0.95 + \text{ruido}$), el valor puede superar 1.0. La función `np.clip(..., 0.0, 1.0)` bloquea este desbordamiento, evitando fallos matemáticos en Keras.
- **Independencia del Ground Truth:** Las rotaciones se aplican a ambos recortes, pero el ruido y el brillo modifican únicamente la imagen original de entrada, manteniendo la máscara del médico como un mapa binario puro.

- 6) **Adaptación Tensorial (Expand Dims):** OpenCV es pragmático: al leer la máscara en escala de grises, devuelve una matriz bidimensional plana ($128, 128$). Sin embargo, las capas convolucionales de Keras son rigurosas y exigen siempre una estructura tridimensional (*Alto, Ancho, Canales*). Si se introduce la matriz cruda, la ecuación genera un error de incompatibilidad de formas (*Shape mismatch*). Para solucionarlo, se emplea la instrucción `np.expand_dims(..., axis=-1)`, que inventa una dimensión vacía al final del tensor, transformando la forma a $(128, 128, 1)$ y envolviendo los datos exactamente como Keras requiere.

Algoritmo 1: Lógica del método `__getitem__` del `DataGenerator`

Input: Índice del lote idx

Output: Matrices X (imágenes) e Y (máscaras)

```

lote ← rutas[idx · B : (idx + 1) · B]
para cada (r_img, r_mask) en lote hacer
    img ← leer_y_CLAHE(r_img)/255,0
    mask ← leer_mascara(r_mask)/255,0
    x,y ← coord_seguras_aleatorias(img, P=128)
    p_img ← img[y : y+P, x : x+P]
    p_mask ← mask[y : y+P, x : x+P]
    rand() > 0,5 p_img ← voltear_y_rotar(p_img)
    p_mask ← voltear_y_rotar(p_mask)
    rand() > 0,5
    p_img ← añadir_ruido_y_brillo(p_img)
    p_img ← clip(p_img, 0,0, 1,0)
    X += p_img; Y += expand_dims(p_mask)

```

devolver X, Y

B. Arquitectura de la red: U-Net

La red implementada sigue la arquitectura **U-Net** [3]. Al tener conexiones de salto que bifurcan el flujo de datos, no puede construirse con `keras.Sequential` (solo permite capas en línea recta). Por eso se usa la **API Funcional de Keras**, que permite conectar las capas de forma flexible.

La estructura general de la arquitectura se puede observar en la Fig. 1, extraída del artículo original de Ronneberger et al. [3]. En ella se aprecia claramente el camino de contracción

(izquierda), el camino de expansión (derecha) y las conexiones de salto en gris que unen ambas ramas.

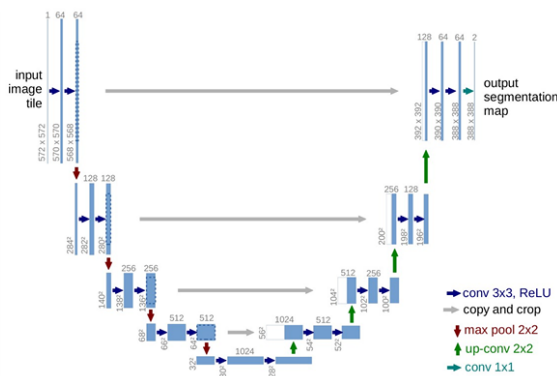


Fig. 1. Arquitectura U-Net original [3]. Las flechas azules representan convoluciones con ReLU, las rojas el *max pooling*, las verdes el *up-conv* y las grises las conexiones de salto (*copy and crop*).

La red recibe parches de (128, 128, 3) y se estructura en:

- **Bloque de doble convolución:** Dos filtros de 3×3 con activación ReLU y *padding same*. La primera detecta patrones simples (bordes) y la segunda detecta patrones más complejos sobre esos resultados, como sería el caso de esquinas, cruces, etc. Se usa *he_normal* como inicializador, óptimo para ReLU.
- **Encoder:** En cada nivel, doble convolución seguida de *MaxPooling* 2×2 . El número de filtros se duplica en cada nivel (16, 32, 64, 128, 256).
- **Conexiones de salto:** Antes de cada *pooling* se guarda una copia de las activaciones que el decoder recuperará, combinando el detalle espacial del encoder con el contexto semántico del decoder.
- **Decoder:** *UpSampling* 2×2 , concatenación con la copia del encoder y doble convolución. Los filtros se reducen a la mitad en cada nivel.
- **Capa de salida:** Convolución 1×1 con activación sigmoide, produciendo un valor de probabilidad entre 0 y 1 por píxel.

C. Entrenamiento con validación cruzada de 5 pliegues

Con solo 20 imágenes de entrenamiento, una división única en entrenamiento y validación sería demasiado inestable y dependiente del azar. Para evitar este problema se aplica **validación cruzada con 5 pliegues** (*5-Fold Cross-Validation*) [14]. Las 20 imágenes se dividen en 5 grupos de 4; en cada iteración se entrena con 16 imágenes y se valida con las 4 restantes, rotando el grupo de validación en cada paso. Al finalizar las 5 iteraciones se obtienen 5 modelos independientes que en la fase de evaluación se combinan en un *ensemble*.

Cada uno de los 5 modelos se entrena desde cero de forma idéntica con los siguientes hiperparámetros y herramientas de control:

- **Optimizador:** Adam [13] con una tasa de aprendizaje inicial de 0,0001. Se eligió porque es un método de

optimización adaptativo, lo que significa que no aplica la misma velocidad de aprendizaje a todos los pesos de la red de forma fija. Adam funciona combinando dos ideas clave: por un lado, utiliza el “momentum” o inercia, que aprovecha la tendencia de los pasos anteriores para que el entrenamiento avance más rápido y no se quede atascado en zonas difíciles; y por otro lado, calcula un ritmo de aprendizaje independiente para cada parámetro individual según la frecuencia con la que cambia. Al configurar una tasa inicial pequeña de 0,0001, nos aseguramos de que la U-Net aprenda con cuidado y haga ajustes muy minuciosos en las venas más finas, evitando dar saltos bruscos que estropeen lo aprendido.

- **Función de pérdida:** BCE-DICE Loss combinada. Suma la entropía cruzada binaria píxel a píxel con el error del DICE global, haciendo el entrenamiento mucho más estable frente al desbalanceo de clases.
- **Batch size:** 4 parches de 128×128 píxeles por paso. Permite que el modelo procese datos constantemente sin desbordar la memoria RAM.
- **Épocas:** Se configura un máximo de 200 épocas. Como los ingenieros no somos adivinos y no sabemos cuándo alcanzará la red su máximo rendimiento, no dejamos el final a ciegas y delegamos el control en tres funciones de ayuda o *callbacks* [9]:
 - **EarlyStopping:** Actúa como un freno de mano automático para evitar el sobreajuste (*overfitting*). Configurado con una paciencia de 25 épocas, vigila constantemente la nota del examen de prueba (*val_dice_coef*). Si el modelo pasa 25 épocas seguidas sin superar su propio récord, detiene el entrenamiento de golpe y restaura los mejores pesos guardados.
 - **ModelCheckpoint:** Funciona como nuestro sistema de copias de seguridad. Cada vez que la red consigue una puntuación récord de DICE en el grupo de validación, este vigilante sobrescribe el archivo *.keras* en el disco duro. Esto nos garantiza que el modelo final guardado sea el mejor de todo el proceso y no simplemente el de la última época ejecutada.
 - **ReduceLROnPlateau:** Introduce una estrategia de aprendizaje granular. Si la pérdida de validación (*val_loss*) se queda estancada durante 7 épocas seguidas, este callback reduce la tasa de aprendizaje a la mitad (multiplica por 0,5). Esto permite al optimizador Adam dar pasos mucho más pequeños y cuidadosos en las etapas avanzadas, ayudando a la red a salir de valles locales y afinar la detección de capilares.

El proceso completo genera 5 archivos independientes (*fold_1.keras* a *fold_5.keras*). Durante la inferencia en el script de evaluación, las predicciones de estos 5 modelos se promedian. Esto reduce drásticamente la varianza, ya que los puntos débiles o dudas de un pliegue concreto se compensan con los aciertos de los otros cuatro.

Algoritmo 2: Flujo general del bucle de entrenamiento con validación cruzada

Input: 20 Imágenes de ojo en color, 20 Máscaras Ground Truth

Output: 5 Modelos entrenados guardados en formato .keras

$rutas_img, rutas_gt \leftarrow obtener_y_ordenar_rutas()$
 $kf \leftarrow Configurar_KFold(pliegues = 5, barajar = True)$

para cada (idx_train, idx_val) **en** $kf.split(rutas_img)$ **hacer**

```
    fold  $\leftarrow$  número_iteración_actual  
    gen_train  $\leftarrow$  DataGenerator(rutas_img[idx_train]  $\times$   
        10, batch = 4)  
    gen_val  $\leftarrow$   
        DataGenerator(rutas_img[idx_val], batch = 4)  
    modelo  $\leftarrow$  construir_unet(input_shape =  
        (128, 128, 3))  
    modelo.compile(optimizer = Adam( $1e^{-4}$ ), loss =  
        bce_dice_loss)  
    callbacks  $\leftarrow$  [  
        EarlyStopping(monitor =  
            val_dice_coef, patience = 25),  
  
        ModelCheckpoint(ruta = "fold_" + fold + ".keras"),  
        ReduceLROnPlateau(monitor =  
            val_loss, patience = 7)  
    ]  
    modelo.fit(gen_train, validation_data =  
        gen_val, callbacks = callbacks, epochs = 200)
```

D. Postprocesado de la predicción

Una vez el *ensemble* genera el mapa de probabilidades se aplican tres pasos:

- 1) **Umbralización con Otsu:** El método de Otsu [12] calcula automáticamente el mejor punto de corte para cada imagen. Se aplica además un ajuste de $-0,04$ para recuperar vasos finos que quedarían por debajo del umbral.
- 2) **Cierre morfológico:** Para sellar pequeñas roturas en los vasos se aplica un cierre morfológico [15] (dilatación seguida de erosión) con un elemento estructurante elíptico de 3×3 px, que respeta mejor la forma curva de los vasos que un kernel cuadrado.
- 3) **Filtrado de componentes pequeñas:** Se eliminan los grupos de píxeles blancos conectados con área inferior a 2 px, limpiando el ruido sin afectar a los vasos reales.

E. Proceso de evaluación y postprocesado

La evaluación del conjunto de test no consiste simplemente en pasar las imágenes por la red neuronal y obtener un resultado directo. Las redes neuronales devuelven un mapa de probabilidades continuo (valores decimales entre 0 y 1 por cada píxel). Convertir esa probabilidad en una máscara binaria

de alta fidelidad clínica requiere combinar el *Deep Learning* con técnicas clásicas de Visión por Computador.

Es importante destacar que todos los hiperparámetros de esta fase (tamaños de kernel, umbrales y áreas) no fueron elegidos al azar, sino que fueron calibrados empíricamente tras múltiples pruebas en un entorno de experimentación aislado (el cuaderno `evaluation_test.ipynb`) hasta dar con la configuración que maximizaba el rendimiento.

El proceso final, codificado en el script `evaluate.py` y resumido en el Alg. 3, sigue una arquitectura de inferencia de cuatro etapas para cada imagen del conjunto de test:

- 1) **Ensemble y Test-Time Augmentation (TTA) por Medias:** Para generar el primer mapa de probabilidades, no usamos un solo modelo ni le pasamos la imagen una sola vez. Se iteran los 5 modelos entrenados en el *K-Fold*. A cada modelo se le pasa la imagen original, la imagen volteada horizontalmente y la imagen volteada verticalmente. Posteriormente, las predicciones se "des-voltean" a su posición original y se calcula la media matemática de todas las vistas. Esta técnica es fundamental para estabilizar la predicción: si la red tiene dudas en una zona oscura y "alucina" una mancha de ruido en una de las vistas, al promediarla con los ceros de las otras vistas, el error se diluye y desaparece.
- 2) **Binarización Dinámica (Otsu) y Compensación:** Un umbral estático (por ejemplo, considerar vaso todo lo mayor a 0,5) asume erróneamente que todas las fotografías tienen la misma iluminación. Para solucionarlo, aplicamos el algoritmo de Otsu de OpenCV, que analiza el histograma de cada imagen individual y calcula de forma automática el mejor punto de corte. Sin embargo, Otsu tiende a ser conservador. Para rescatar los micro-capilares de muy bajo contraste, implementamos una compensación u *offset* de $-0,040$. Restar este 4% al umbral óptimo fuerza a la máquina a ser ligeramente más permisiva, mejorando notablemente la tasa de detección (Recall).
- 3) **Morfología Matemática Elíptica:** Al usar un umbral tan afinado, algunos capilares pueden quedar fragmentados. Para solucionarlo, se aplica una operación de Cierre Morfológico (una dilatación seguida de erosión). El detalle técnico clave es el uso de un **kernel elíptico** de 3×3 píxeles (`MORPH_ELLIPSE`). Esto obedece a la realidad biológica del problema: las venas son estructuras orgánicas, curvas y tubulares. Usar un filtro cuadrado clásico introduciría aristas y un "efecto sierra", mientras que la elipse sella las micro-roturas respetando el grosor y la topología anatómica original.
- 4) **Limpeza (Ruido Sal y Pimienta) y Enmascarado:** Se ejecuta un analizador de componentes conexos que busca y elimina cualquier grupo de píxeles blancos aislados cuya área sea inferior a 2 píxeles, limpiando el ruido residual sin borrar vasos reales. Finalmente, la predicción se multiplica por la máscara FOV para asegurar que todo lo que caiga fuera del globo ocular sea estrictamente negro. Tras esto, se calcula el coeficiente

DICE contra las máscaras de los dos expertos y se promedia.

Algoritmo 3: Proceso de evaluación combinando TTA y Visión Clásica

Input: Lista de 5 modelos entrenados, 20 imágenes de test

Output: DICE medio global frente a ambos expertos

para cada *img* **en** imágenes **hacer**

original $\leftarrow$ leer_y_CLAHE(*img*)/255,0

mask_fov $\leftarrow$ leer_mascara_fov(*img*)

gt₁, *gt₂* $\leftarrow$

leer_verdad_terreno(*img*, experto 1 y 2)

predicciones $\leftarrow$ []

para cada *modelo* **en** modelos **hacer**

p $\leftarrow$ predecir_por_parches(*modelo*, *original*)

p_h $\leftarrow$

predecir_por_parches(*modelo*, voltear_h(*original*))

p_v $\leftarrow$

predecir_por_parches(*modelo*, voltear_v(*original*))

predicciones +=

(*p* + voltear_h(*p_h*) + voltear_v(*p_v*))/3

mapa_prob $\leftarrow$ media(*predicciones*)

umbral $\leftarrow$ otsu(*mapa_prob*) - 0,040

mask_bin $\leftarrow$ *mapa_prob* > *umbral*

mask_bin $\leftarrow$

cierre_morfologico(*mask_bin*, kernel_eliptico = 3)

mask_bin $\leftarrow$

filtrar_componentes(*mask_bin*, area_min = 2)

mask_bin $\leftarrow$ *mask_bin* $\wedge$ *mask_fov*

dice₁ $\leftarrow$ DICE(*mask_bin*, *gt₁*)

dice₂ $\leftarrow$ DICE(*mask_bin*, *gt₂*)

registrar(*img*, *dice₁*, *dice₂*, *umbral*)

devolver media({(*dice₁* + *dice₂*)/2})

F. Justificación de las decisiones de diseño

A la hora de desarrollar el proyecto, cada línea de código y cada configuración del modelo se eligieron con un motivo claro, buscando el equilibrio entre precisión médica y eficiencia en ordenadores normales. Muchas de estas decisiones estratégicas se tomaron tras analizar diferentes aproximaciones de la comunidad [17], [18] y realizar pruebas empíricas en cuadernos de trabajo independientes:

- **API Funcional de Keras frente a Sequential:** La arquitectura U-Net exige conectar capas de la bajada (encoder) directamente con capas de la subida (decoder) mediante conexiones de salto. Como el modelo tradicional Sequential de Keras solo permite apilar capas en línea recta (una detrás de otra), fue obligatorio utilizar la API Funcional. Esto nos permitió tratar las capas como funciones y utilizar la instrucción

`layers.concatenate` para fusionar los mapas de características espaciales.

- **Inicialización de pesos con He Normal:** Por defecto, las capas convolucionales inicializan sus pesos de forma aleatoria estándar. Sin embargo, al usar la función de activación ReLU en toda la red, existe el riesgo de que los gradientes se vuelvan infinitamente pequeños en las capas más profundas (problema del desvanecimiento del gradiente). Configurar `kernel_initializer='he_normal'` asegura que los pesos iniciales se adapten a la matemática de ReLU, estabilizando el arranque del entrenamiento.
- **Ecualización CLAHE dentro del Generador:** El filtro CLAHE sirve para resaltar objetos ocultos en la oscuridad. Gracias a un cuaderno inicial de pruebas y desarrollo (`exploracion.ipynb`), pudimos comprobar visualmente la estructura y los formatos de las imágenes antes de normalizarlas. Esto nos ayudó a diseñar el paso intermedio de aislar el canal de luminosidad (L) en el espacio LAB, consiguiendo mejorar el contraste de los capilares finos sin alterar los colores originales de la retinografía.
- **Parcheado en lugar de Redimensionado:** El dataset DRIVE tiene imágenes de 565×584 píxeles. Si hubiéramos cambiado el tamaño de las fotos para hacerlas más pequeñas y cuadradas, habríamos aplastado y deformado los vasos capilares (que miden apenas 1 o 2 píxeles de ancho), haciendo que la red fuera incapaz de aprender. El parcheado aleatorio de 128×128 nos permitió conservar la resolución y proporción real de los vasos, además de reducir drásticamente el consumo de memoria RAM.
- **Gestión de entrenamiento mediante Callbacks encadenados:** En lugar de fijar un número arbitrario de épocas arriesgándonos a dejar el modelo a medio entrenar o a quemarlo por sobreajuste, diseñamos un sistema de control reactivo basado en tres callbacks de Keras. Configurar la combinación de `EarlyStopping`, `ModelCheckpoint` y `ReduceLROnPlateau` nos permitió automatizar la gestión del entrenamiento: la red disminuye su velocidad de aprendizaje si se estanca, aborta el bucle si deja de mejorar y asegura la persistencia del mejor estado en el disco duro de forma autónoma.
- **Test-Time Augmentation (TTA) por Medias frente a Máximos:** En el archivo `evaluate.py`, implementamos TTA para reducir las dudas de la red en zonas difíciles. Durante las fases de pruebas en el entorno experimental de `evaluation_test.ipynb`, ensayamos utilizando una política de TTA basada en quedarnos con el valor máximo de probabilidad entre las vistas. Esto fue un fracaso, ya que amplificó el ruido de fondo y tiró el DICE score general abajo hasta un 0,67. Al cambiar el diseño para calcular la media aritmética de las tres vistas (original, volteo horizontal y vertical), conseguimos que el ruido aleatorio se diluyera y que los capilares reales se reforzaran.

- **Kernel Elíptico en la Morfología:** Tras binarizar la imagen con Otsu, aplicamos un cierre morfológico para unir trozos de vasos que hubieran quedado separados. Mediante las pruebas visuales que realizamos en el cuaderno `evaluation_test.ipynb`, decidimos configurar un elemento estructurante elíptico (`cv2.MORPH_ELLIPSE`) de 3×3 píxeles en lugar de uno cuadrado. La justificación es biológica: los vasos sanguíneos son redes tubulares curvas. Un filtro cuadrado deformaría los capilares creando bordes rectos artificiales en forma de sierra, mientras que la elipse sella las roturas respetando la forma orgánica y curva original.

G. Análisis de los obstáculos superados

Durante el transcurso del proyecto nos enfrentamos a varios problemas técnicos y teóricos que amenazaban el rendimiento del sistema o la viabilidad del entrenamiento. Para superar estos obstáculos sin dar palos de ciego, diseñamos varios scripts de pruebas unitarias e integración que sirvieron como evidencias para asegurar el correcto funcionamiento del software:

- 1) **Saturación de la Memoria RAM:** Al intentar trabajar con imágenes médicas de alta resolución y matrices de datos completas, es muy fácil desbordar la memoria de un ordenador normal. Superamos este obstáculo gracias al diseño diferido (*Lazy Loading*) de la clase `DataGenerator`. Al heredar de `PyDataset`, el código solo procesa lotes pequeños de 4 parches en el instante exacto en que la red va a realizar un paso de entrenamiento, liberando los recursos inmediatamente después. Esto permitió entrenar el modelo de forma fluida.
- 2) **El Engaño de las Clases Desbalanceadas y Validación de Métricas:** Como el fondo negro del ojo ocupa casi el 90% de la imagen, la red descubrió rápidamente que si predecía que todo era fondo negro, su tasa de acierto (*Accuracy*) era altísima, por lo que dejaba de aprender a buscar venas. Superamos este obstáculo eliminando el *Accuracy* como guía y diseñando una función de pérdida híbrida llamada `bce_dice_loss`. Para asegurar que las matemáticas de esta métrica personalizada funcionaban de forma perfecta antes de lanzar el entrenamiento real, programamos un script de pruebas unitarias llamado `test_metrics.py`. Este archivo simula una matriz de juguete de 2×2 píxeles y comprueba de forma aislada que una predicción idéntica arroja un DICE exacto de 1,0 y que una totalmente fallida da un valor cercano a 0,0, blindando la lógica del aprendizaje.
- 3) **Errores de Integración en el Pipeline:** Al juntar por primera vez el generador de parches dinámico con el modelo de red neuronal U-Net, existía un riesgo muy alto de sufrir fallos matemáticos por desajustes en el tamaño y la profundidad de las matrices (*shape mismatch*). Para resolver esto de manera limpia, creamos un script de pruebas de integración llamado

`test_integracion.py`. Este programa nos sirvió de evidencia para verificar el acoplamiento correcto de los componentes; se encarga de extraer un lote de prueba, pasarlo por la red sin entrenar y comprobar visualmente en un gráfico de tres paneles la coherencia del flujo (imagen de entrada, máscara de respuestas y ruido predictivo inicial), confirmando que las dimensiones encajaban perfectamente.

- 4) **Eliminación de ruido sin borrar capilares reales:** Al implementar el umbral adaptativo de Otsu con el margen permisivo de $-0,040$, conseguimos detectar los vasos más finos, pero a costa de generar ruido “sal y pimienta” (píxeles blancos sueltos por el fondo oscuro). El obstáculo surgió al intentar limpiar este ruido: los filtros convencionales borraban el fondo, pero también hacían desaparecer los micro-capilares que tanto nos había costado detectar. Superamos este problema experimentando en el cuaderno `evaluation_test.ipynb` con la librería OpenCV. Diseñamos una solución en dos fases: primero, el cierre morfológico elíptico une los capilares fragmentados para protegerlos. A continuación, aplicamos un analizador de componentes conexos (`cv2.connectedComponentsWithStats`) ajustado específicamente para borrar solo aquellos grupos de píxeles sueltos cuyo tamaño fuera menor a 2 píxeles. Esto nos permitió limpiar por completo las manchas falsas del fondo sin perder las venas más finas.
- 5) **Pérdida de Capilares en Fotos Oscuras:** Al evaluar el conjunto de test, vimos que las imágenes con peor iluminación (como la imagen `15_test`) daban resultados muy malos si usábamos un umbral de corte fijo de 0,5, ya que la red asignaba probabilidades bajas a los vasos oscuros. Resolvimos este obstáculo eliminando los cortes fijos e implementando el algoritmo adaptativo de Otsu, que calcula el mejor umbral foto por foto. Además, añadimos un ajuste fino de $-0,040$ a ese umbral dinámico; este margen de seguridad actúa como un colchón que obliga al software a rescatar las venas más tenues y periféricas de la retina, salvando la puntuación DICE en los casos médicos más complejos.

VI. RESULTADOS

A. Configuración experimental

El entrenamiento y la evaluación se realizaron con Python 3, TensorFlow [5] y Keras 3 [6] sobre una GPU Tesla T4. Las 20 imágenes del conjunto de test no participaron en ninguna fase de entrenamiento ni validación. Para la predicción final se empleó el *ensemble* de los 5 modelos con *test-time augmentation* (TTA): se predice también sobre la imagen volteada horizontal y verticalmente, promediando las tres predicciones.

B. Resultados obtenidos

Todas las métricas se calcularon dentro del área definida por la máscara FOV, siguiendo la norma de evaluación de DRIVE 2004. La Tabla II recoge los valores medios globales y la Tabla III los resultados individuales por imagen.

TABLA II
RESULTADOS MEDIOS SOBRE EL CONJUNTO DE TEST DE DRIVE 2004

Evaluación	DICE medio	Precisión	Recall
Frente al 1.er experto	0,7692	0,8695	0,7016
Frente al 2.º experto	0,7799		
Media total	0,7746	0,8695	0,7016

Junto al DICE se reportan también la **precisión** y el **recall** como métricas complementarias extraídas de la **matriz de confusión** del modelo, lo que permite interpretar mejor su comportamiento. La precisión $Prec = \frac{TP}{TP+FP}$ mide qué proporción de los píxeles marcados como vaso por el modelo son realmente vasos: un valor alto indica pocas falsas alarmas (falsos positivos). El recall $Rec = \frac{TP}{TP+FN}$ mide qué proporción de los vasos reales ha detectado el modelo: un valor alto indica que se pierden pocos vasos (falsos negativos). Ambas métricas se calculan también dentro del área FOV. El DICE, como se definió en la Ec. 1, es precisamente la media armónica de estas dos medidas, por lo que un DICE alto solo es posible si tanto la precisión como el recall son simultáneamente elevados. Con una precisión de 0,8695 y un recall de 0,7016, el modelo tiende a ser conservador: marca con alta fiabilidad los píxeles que clasifica como vaso, pero deja sin detectar algunos capilares finos, lo que explica el mayor peso de los falsos negativos sobre los falsos positivos en el resultado global.

TABLA III
RESULTADOS INDIVIDUALES POR IMAGEN DE TEST

Imagen	Umbral Otsu	DICE Exp1	DICE Exp2	Media
01_test	0,3914	0,7801	0,7898	0,7850
02_test	0,4031	0,8244	0,8307	0,8276
03_test	0,3757	0,7676	0,7924	0,7800
04_test	0,4031	0,7912	0,7969	0,7940
05_test	0,3875	0,7847	0,8094	0,7970
06_test	0,3875	0,7637	0,7607	0,7622
07_test	0,3953	0,7682	0,7615	0,7648
08_test	0,3757	0,7334	0,7318	0,7326
09_test	0,3718	0,7449	0,7389	0,7419
10_test	0,3914	0,7665	0,7754	0,7710
11_test	0,3992	0,7616	0,7807	0,7711
12_test	0,3875	0,7771	0,7847	0,7809
13_test	0,3953	0,7655	0,7713	0,7684
14_test	0,3914	0,7641	0,7626	0,7634
15_test	0,3875	0,7505	0,7310	0,7408
16_test	0,3953	0,7713	0,8063	0,7888
17_test	0,3757	0,7419	0,7727	0,7573
18_test	0,3914	0,7688	0,8148	0,7918
19_test	0,3953	0,7917	0,8118	0,8017
20_test	0,3835	0,7677	0,7748	0,7712
Media	0,3886	0,7692	0,7799	0,7746

C. Análisis visual de predicciones

Las siguientes figuras muestran tres casos representativos del conjunto de test. Cada imagen presenta en cuadrícula: fotografía original del ojo (arriba izquierda), segmentación del Experto 1 (arriba derecha), segmentación del Experto 2 (abajo izquierda) y predicción del modelo (abajo derecha).

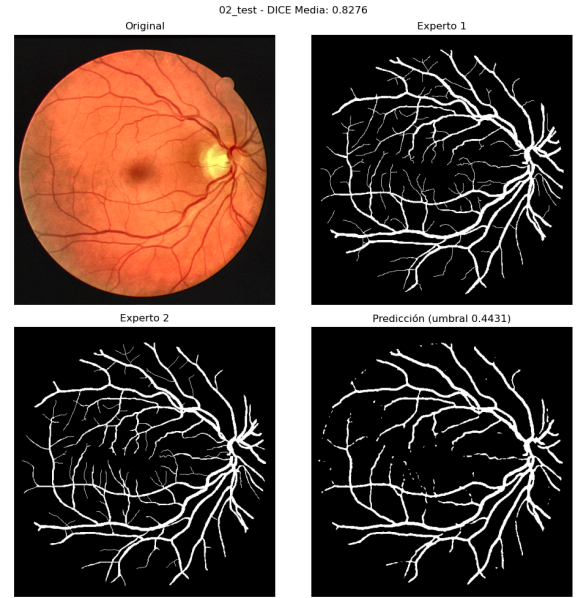


Fig. 2. 02_test (DICE 0,8276 — mejor resultado).

La Fig. 2 corresponde a la imagen 02_test, que obtuvo el mejor DICE del conjunto (0,8276). La fotografía original presenta un ojo con buen contraste y vasos bien definidos. La predicción del modelo se alinea con gran precisión con las anotaciones de ambos expertos: los vasos principales y sus ramificaciones quedan marcados con contornos limpios y sin apenas ruido de fondo. Este caso ilustra el escenario ideal en que el modelo trabaja, con iluminación uniforme y alta nitidez.

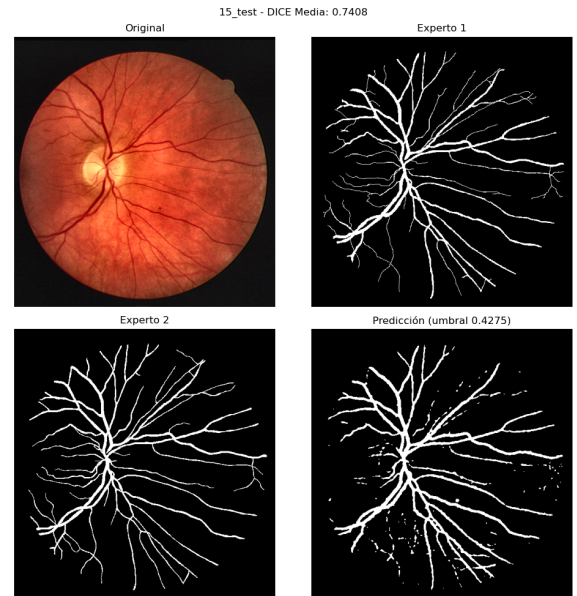


Fig. 3. 15_test (DICE 0,7408 — resultado intermedio).

La Fig. 3 muestra la imagen 15_test, con un DICE de 0,7408, representativa del caso intermedio. El modelo detecta correctamente los vasos de mayor calibre, pero en la zona inferior derecha aparece un exceso de píxeles blancos (falsos

positivos) que no corresponden a vasos reales. Esto coincide con una zona de la fotografía donde el contraste es menos uniforme, lo que dificulta la discriminación del modelo.

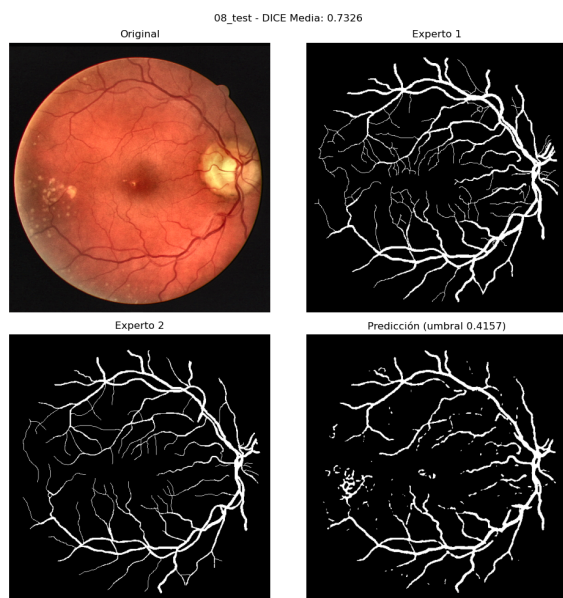


Fig. 4. 08_test (DICE 0,7326 — peor resultado).

La Fig. 4 corresponde a la imagen 08_test, con el peor DICE del conjunto (0,7326). La fotografía original muestra una retina con manchas claras irregulares (exudados) provocadas por la retinopatía diabética, que el modelo confunde con vasos generando una zona central con mucho ruido de falsos positivos. Además, la red de vasos finos en el cuadrante inferior izquierdo queda parcialmente sin detectar. Este caso pone de manifiesto la principal limitación del modelo: su sensibilidad a patologías que alteran visualmente el fondo de ojo, algo que solo podría resolverse con más datos de entrenamiento que incluyan este tipo de casos.

D. Análisis global de los resultados

El modelo superó el umbral mínimo de DICE 0,75 en 17 de las 20 imágenes, con una media global de 0,7746. Las tres imágenes que no alcanzaron el umbral individualmente (08, 09 y 15) presentaban condiciones fotográficas o patológicas más complicadas, como las lesiones por exudado visibles en 08_test. La alta precisión (0,8695) indica que cuando el modelo marca un píxel como vaso, acierta en la gran mayoría de los casos. El recall más bajo (0,7016) confirma que el modelo es algo conservador con los vasos capilares más finos, que son los más difíciles de detectar incluso para un observador humano. El modelo obtiene además un DICE ligeramente mayor frente al segundo experto (0,7799) que frente al primero (0,7692), lo que sugiere que el estilo de anotación del segundo se alinea algo mejor con las predicciones generadas.

E. Exposición de hallazgos principales

A lo largo del desarrollo de este proyecto, hemos realizado varios descubrimientos importantes que van más allá del

simple hecho de programar la red neuronal. Estos hallazgos nos han ayudado a entender cómo se comporta la inteligencia artificial al enfrentarse a un problema médico real:

- **El aumento de datos es más importante que el tamaño de la red:** Al principio, pensábamos que el éxito del modelo dependería de hacer una red U-Net muy profunda y compleja. Sin embargo, descubrimos que el verdadero cuello de botella era la falta de datos. Aplicar giros, ruidos y cambios de brillo "al vuelo" a nuestras únicas 20 imágenes hizo que la red aprendiera mucho mejor. Nos dimos cuenta de que es preferible tener un modelo sencillo con datos muy variados, que un modelo complejísimo que simplemente se aprende de memoria las mismas 20 fotos.
- **El poder de la evaluación múltiple (TTA) como filtro de ruido:** Uno de los mayores hallazgos a nivel de código fue comprobar cómo se reducía el ruido al hacer predicciones. Si a la red le pasábamos una zona oscura, a veces se inventaba una mancha blanca (falso positivo). Pero descubrimos que si le pasábamos esa misma imagen volteada como un espejo, la red ya no se inventaba esa mancha. Al hacer la media matemática de las distintas vistas, el error simplemente desaparecía. Fue un hallazgo clave porque nos permitió limpiar la imagen sin tener que programar filtros agresivos que borrarán las venas finas.
- **La importancia de una función de pérdida híbrida (BCE-DICE):** Uno de nuestros hallazgos más críticos fue descubrir que confiar únicamente en la métrica DICE para entrenar la red no era suficiente. Al principio, la red tendía a quedarse estancada intentando minimizar el error de solapamiento global, pero ignoraba los detalles finos de los capilares más pequeños. Descubrimos que, al combinar el DICE (que mide el solapamiento global) con la *Binary Cross-Entropy* (que obliga a la red a "decidir" correctamente píxel a píxel cada punto del mapa), forzamos al modelo a aprender una doble lección: la forma general del vaso y la precisión extrema del contorno. Este hallazgo fue el punto de inflexión donde el modelo dejó de ser "conservador" y empezó a capturar las estructuras más delicadas que antes se perdían en el ruido de fondo.
- **La rigidez de los umbrales manuales:** Nos dimos cuenta de que establecer reglas fijas en medicina no funciona. Al principio, decíamos: "todo lo que tenga más de un 0,5 de probabilidad es vena". Esto funcionaba genial en fotos claras, pero fracasaba estrepitosamente en fotos oscuras. El hallazgo fue entender que cada paciente y cada foto necesitan su propio límite. Dejar que el algoritmo de Otsu decidiera ese límite por nosotros, y darle simplemente un "empujoncito" restándole 0,040, fue la clave para que el código se adaptara a la iluminación de cualquier retinografía.
- **Divergencia de convergencia entre pliegues:** Gracias al uso de los callbacks adaptativos, descubrimos que cada uno de los 5 pliegues del *K-Fold* se comportaba de forma

completamente única. Algunos pliegues alcanzaban su mejor DICE score en la época 60 y empezaban a empeorar, mientras que otros necesitaban estirarse hasta la época 110 para exprimir su rendimiento. Este hallazgo nos demostró empíricamente que fijar un número de épocas estático para todos los modelos por igual habría provocado que algunos pliegues sufrieran de subentrenamiento y otros de sobreajuste, justificando por completo nuestra decisión de diseño.

- **La Inteligencia Artificial y la duda humana:** El hallazgo más sorprendente a nivel clínico fue entender la variabilidad inter-observador. Cuando analizamos las imágenes donde el modelo sacaba peor nota (como la `15_test`), vimos que no era porque el código estuviera mal hecho, sino porque en esas zonas tan oscuras ni siquiera los dos médicos expertos estaban de acuerdo en dónde pintar la vena. Descubrimos que cuando la red neuronal comete errores en los capilares más tenues, en realidad está llegando al mismo límite de duda y ambigüedad que tiene un ojo humano profesional.
- **La sensibilidad ante patologías desconocidas:** En imágenes como la `08_test`, descubrimos que nuestro modelo es muy ingenuo ante enfermedades que no ha visto lo suficiente. Al haber manchas brillantes provocadas por la propia diabetes, el modelo se confundía e intentaba marcar bordes de venas donde no los había. Esto nos demostró que una inteligencia artificial es tan buena como los datos con los que se entrena: si no le enseñamos suficientes ejemplos de ojos enfermos con manchas, no sabrá ignorarlas en el mundo real.

VII. CONCLUSIONES

En este trabajo se ha diseñado y entrenado una red neuronal con arquitectura U-Net capaz de segmentar automáticamente los vasos sanguíneos en imágenes de retina del dataset DRIVE 2004. El sistema cubre todo el flujo de trabajo: desde la carga eficiente con un generador por lotes que implementa *Lazy Loading*, pasando por el preprocesado con CLAHE y el aumento de datos dinámico mediante volteos y perturbaciones de brillo, hasta el entrenamiento con validación cruzada de 5 pliegues y el postprocesado con umbralización adaptativa de Otsu y filtrado morfológico. El modelo final alcanza un DICE medio de 0,7746, superando el mínimo exigido de 0,75 y situándose en un rango de rendimiento comparable al nivel de acuerdo entre los propios anotadores humanos del dataset.

Las decisiones técnicas adoptadas tuvieron un impacto directo y medible en los resultados. Usar `PyDataset` para la carga diferida permitió entrenar en hardware convencional sin problemas de memoria RAM. Aplicar CLAHE antes de normalizar mejoró la detección de vasos en zonas oscuras de la retina. El *ensemble* de 5 modelos con *test-time augmentation* redujo la varianza de las predicciones respecto a usar un único modelo. El ajuste del umbral de Otsu con un margen negativo de $-0,04$ recuperó vasos finos que una umbralización fija en $0,5$ habría perdido. Y el uso de un kernel elíptico en el cierre morfológico selló las roturas en los vasos respetando mejor su

forma curva que un kernel cuadrado clásico. Cada una de estas decisiones, aunque pueda parecer un detalle menor, contribuyó a la mejora del DICE final.

El análisis visual de los casos extremos aporta una perspectiva cualitativa valiosa que los números por sí solos no transmiten. La Fig. 2 demuestra que el modelo es capaz de trazar con precisión redes vasculares complejas cuando las condiciones fotográficas son favorables. La Fig. 3 muestra que el sistema mantiene un rendimiento aceptable incluso con cierta irregularidad en el contraste. Y la Fig. 4 revela con claridad la principal limitación del modelo: la presencia de exudados y lesiones provocadas por la retinopatía diabética genera falsos positivos que el modelo no puede distinguir de vasos reales, ya que el conjunto de entrenamiento no incluye suficientes ejemplos de este tipo de patología.

Una de las conclusiones más importantes que nos llevamos de este desarrollo es haber descubierto que la verdadera clave para mejorar las estadísticas del modelo no estaba tanto en el diseño de la propia red neuronal, sino en la calidad del preprocesado y del postprocesado. De hecho, la inmensa mayoría del tiempo que pasamos programando, haciendo pruebas y experimentando no la invertimos modificando la arquitectura del modelo, sino refinando el comportamiento de los datos dentro de `generator.py` y puliendo la salida final en `evaluate.py`.

Más allá de los resultados numéricos, este proyecto ha permitido comprender de forma práctica el ciclo completo de desarrollo de un sistema de inteligencia artificial aplicado a un problema médico real. La diferencia entre clasificación y segmentación, el papel de las conexiones de salto en la U-Net, la importancia de evaluar solo dentro del campo de visión o el impacto del umbral de binarización en el DICE final son conceptos que solo se asimilan verdaderamente al enfrentarse a un dataset con sus propias particularidades y limitaciones. El hecho de que el modelo falle precisamente en la imagen con más patología visible resulta también una lección valiosa: la calidad y diversidad del conjunto de entrenamiento es, en última instancia, el factor más determinante en el rendimiento de cualquier sistema de *deep learning*.

REFERENCIAS

- [1] M. Vakalopoulou et al., "Deep learning: basics and convolutional neural networks (CNN)," *Machine Learning for Brain Disorders*, O. Colliot, Ed., HAL, 2023. [En línea]. Disponible: <https://hal.science/hal-03957224v2>
- [2] D. Vélez, "¡Redes Neuronales CONVOLUCIONALES! ¿Cómo funcionan?" YouTube, 2018. [En línea]. Disponible: <https://www.youtube.com/watch?v=V8j1oENVz00>
- [3] O. Ronneberger, P. Fischer y T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *arXiv*, 2015. [En línea]. Disponible: <https://arxiv.org/abs/1505.04597>. Repositorio oficial: <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>
- [4] S. Abdusamatov, "Biomedical Image Segmentation with U-Net," Kaggle, 2022. [En línea]. Disponible: <https://www.kaggle.com/code/saidislombek/biomedical-image-segmentation-with-u-net>
- [5] Google, "Introducción a TensorFlow," TensorFlow, 2024. [En línea]. Disponible: <https://www.tensorflow.org/learn?hl=es-419>
- [6] Google, "Guía de Keras 3," TensorFlow, 2024. [En línea]. Disponible: <https://www.tensorflow.org/guide/keras?hl=es-419>

- [7] J. Staal et al., "DRIVE: Digital Retinal Images for Vessel Extraction," Kaggle, 2004. [En línea]. Disponible: <https://www.kaggle.com/datasets/zionfu/drive2004>
- [8] Team Keras, "PyDataset class API," Keras Documentation, 2024. [En línea]. Disponible: https://keras.io/api/utils/python_utils/#pydataset-class
- [9] Team Keras, "Keras Callbacks API," Keras Documentation, 2024. [En línea]. Disponible: <https://keras.io/api/callbacks/>
- [10] Wikipedia contributors, "Coeficiente de Sørensen-Dice," Wikipedia, La enciclopedia libre. [En línea]. Disponible: https://es.wikipedia.org/wiki/Coeficiente_de_Sorensen-Dice
- [11] OpenCV, "Histogram Equalization (CLAHE)," OpenCV Documentation. [En línea]. Disponible: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
- [12] Código en Llamas, "Algoritmo de Otsu para binarización de imágenes." [En línea]. Disponible: <https://codigoenllamas.com/algoritmo-de-otsu>
- [13] Team Keras, "Adam Optimizer," Keras Documentation. [En línea]. Disponible: <https://keras.io/api/optimizers/adam/>
- [14] Scikit-learn developers, "sklearn.model_selection.KFold," Scikit-learn Documentation. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html
- [15] OpenCV, "Morphological Transformations," OpenCV Documentation. [En línea]. Disponible: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
- [16] H. Mrabet, "Segmentation (U-Net on DRIVE)," Kaggle. [En línea]. Disponible: <https://www.kaggle.com/code/hasnamrabet/segmentation>
- [17] N. Ahmed, "Retina Blood Vessel Segmentation using UNet," Kaggle. [En línea]. Disponible: <https://www.kaggle.com/code/nasifahmed99/retina-blood-vessel-segmentation-using-unet>
- [18] HiddenNeuron, "U-Net DRIVE," Kaggle. [En línea]. Disponible: <https://www.kaggle.com/code/hiddenneuron/u-net-drive>
- [19] GeeksforGeeks, "CLAHE Histogram Equalization in OpenCV," GeeksforGeeks. [En línea]. Disponible: <https://www.geeksforgeeks.org/python/clahe-histogram-equalization-opencv/>